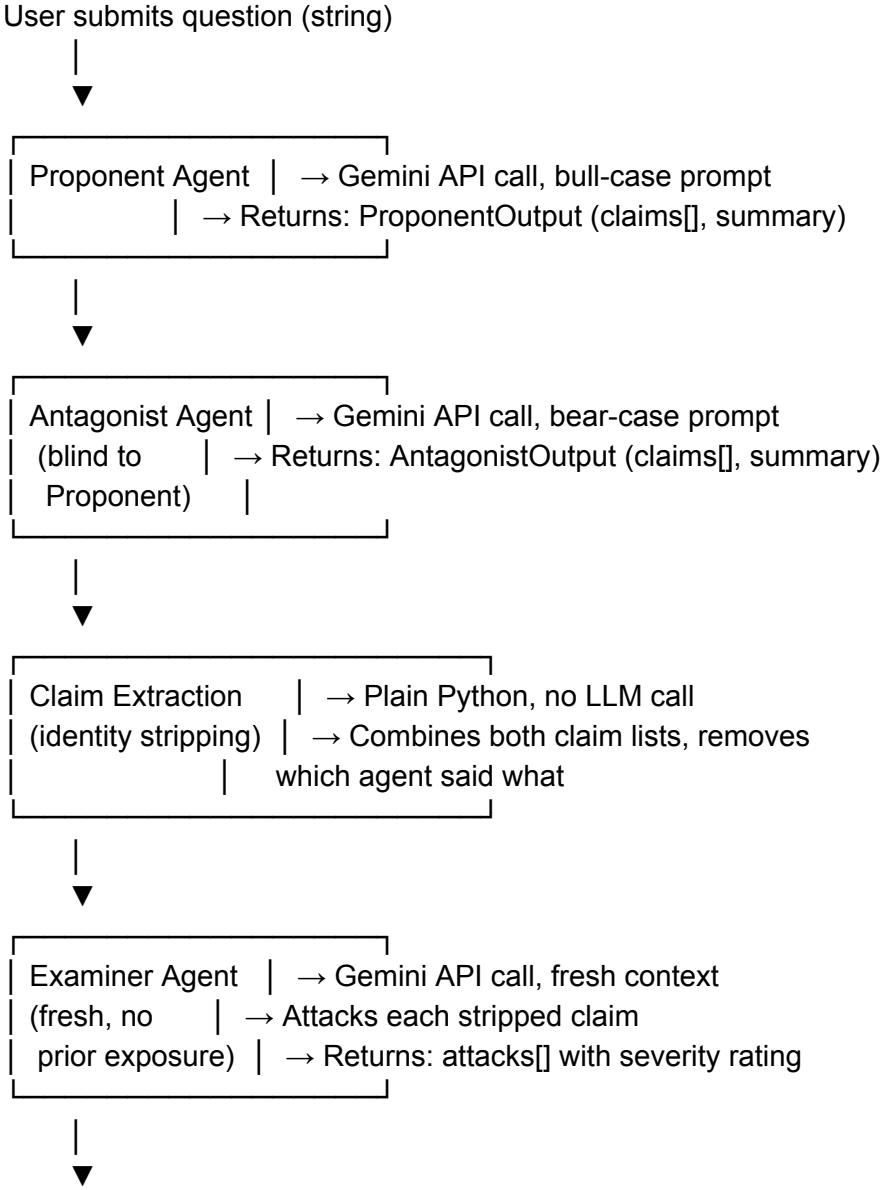
Tribunal — Design Doc

1. System Overview

Tribunal is a sequential pipeline, not a conversational loop. Each stage is a pure function: input in, structured JSON out, validated against a Pydantic schema before moving to the next stage. No stage has open-ended back-and-forth.

2. v0 Architecture



Output: { bull_case, bear_case, flagged_attacks }

3. Key Design Decisions (and why)

Why blind parallel drafting (Proponent + Antagonist never see each other)? Prevents anchoring. If Antagonist saw Proponent's case first, its "bear case" would just be reactive rebuttal, not independent reasoning — narrower and less rigorous.

Why is the Examiner a separate, third agent — not the Antagonist wearing a second hat? If Antagonist cross-examines Proponent's claims using the same agent that already built the bear case, it's evaluating claims through the lens of arguments it already committed to — not fresh scrutiny. A fresh agent with no prior stake in either side produces more honest attacks.

Why strip identity before cross-examination? Prevents the Examiner from attacking claims differently based on "which side" they came from. It should attack weak reasoning wherever it appears, not perform adversarial theater aligned to a side.

Why is claim extraction code, not an LLM call? It's a deterministic transformation (combine two lists, remove a label) — using an LLM for this adds cost, latency, and a new failure point for zero added value. Reserve LLM calls for tasks that genuinely need reasoning.

Why does the Examiner rate severity itself (minor/moderate/fatal) instead of always calling things "fatal"? Instructing an agent to "find fatal flaws" creates a demand characteristic — it'll manufacture severity to satisfy the role even when the real finding is minor. Self-rated severity keeps the system honest.

4. Data Contracts (Pydantic Schemas)

```
class ClaimCategory(str, Enum):
    empirical = "empirical"
    logical = "logical"
    strategic = "strategic"

class Claim(BaseModel):
    id: str
    text: str
    category: ClaimCategory
    confidence: float  # 0.0-1.0

class ProponentOutput(BaseModel):
    claims: List[Claim]
    summary: str

# AntagonistOutput mirrors ProponentOutput (same shape, opposite framing)
```

```
class Severity(str, Enum):
    minor = "minor"
    moderate = "moderate"
    fatal = "fatal"

class Attack(BaseModel):
    claim_id: str
    attack_type: str  # "assumption" | "logical_leap" | "missing_evidence"
    severity: Severity
    rationale: str

class ExaminerOutput(BaseModel):
    attacks: List[Attack]
```

5. v1 Additions (planned, not built)

- **Verification Node:** batched web search call (Tavily) for claims tagged `category: empirical`. Output: `{claim_id, status: VERIFIED|DISPUTED|UNSUBSTANTIATED, source_snippet}`
- **Judge Agent:** takes all claims + attacks + verification tags, produces `DecisionMatrix { summary, surviving_tradeoffs, fatal_flaws }`. Must explicitly downweight DISPUTED/UNSUBSTANTIATED claims per its system prompt — not left to infer.
- **Deadlock detection:** computed in code, not LLM-judged. `deadlock = True` when a claim has `severity: fatal` AND `status: DISPUTED|UNSUBSTANTIATED` AND both Proponent/Antagonist's cases depend on it.

6. Failure Modes to Guard Against

- **Malformed JSON from LLM:** wrap all agent calls in try/except, validate against schema, retry once before failing the pipeline run
- **Rate limiting (free tier):** exponential backoff on 503/429 errors
- **Model deprecation:** pin model version in one place (env var or config, not hardcoded in multiple files) so upgrades are a single change

7. What This Design Deliberately Avoids

- No agent-to-agent conversation loops
- No shared context window across agents (each call is stateless except for what's explicitly passed in)
- No LLM-based routing/orchestration decisions in v0 — the pipeline order is fixed in code